Modeling a More Complex Classification System

Ali Hussain

1 Proposed Pipeline

The proposed pipeline is designed to classify two-dimensional data when the training data does not already have class labels. Most of the work is completed offline on a computer. Only a small lookup step is performed on the embedded device.

First, the training features are scaled using min–max scaling. DBSCAN is then used to discover groups in the unlabelled training data. A Gaussian kernel density estimate (KDE) is fitted separately to each discovered class. Each density map is normalised to its own peak and sampled onto a fixed grid. The grid values are then converted to `uint8` so that each cell requires only one byte.

The embedded device does not run DBSCAN or KDE. Those steps are already completed offline. The device stores the byte tables, the training scaling bounds, the grid size, and the reject threshold.

For a new query point, the device scales the two features using the stored training bounds. It then finds the correct grid cell and reads one byte from each class table. The class with the largest stored value is selected. If the largest value is below the reject threshold, the output is UNKNOWN instead of forcing the point into a class.

Algorithm 1 Table-lookup classification

Require: Raw query point (x, y)
Require: One byte table Q_c for each class
Require: Training scaling bounds
Require: Grid size G
Require: Reject threshold τ
Ensure: Predicted class or UNKNOWN

Scale x and y using the stored training bounds
$col \leftarrow \lfloor \tilde{x} G \rfloor$
$row \leftarrow \lfloor \tilde{y} G \rfloor$
Clip row and col to $[0, G-1]$
for each class c **do**
    $v_c \leftarrow Q_c[row, col]/255$
end for
$\hat{c} \leftarrow \arg\max_c v_c$
if $v_{\hat{c}} < \tau$ **then**
    return UNKNOWN
end if
$p_c \leftarrow v_c / \sum_k v_k$
return $\hat{c}$ and distribution p

2 Application

One possible application for this pipeline is a small wearable motion classifier that recognises different movement patterns from two sensor features.

For this proposed device, I would reserve approximately 4 KB of flash for the classification model. The final lookup tables from this experiment require only 768 bytes, plus 16 bytes for the four scaling bounds, so the model fits comfortably inside this budget.

Labelled training data may not be available because the device could collect motion data during normal use without a person manually identifying every sample. DBSCAN is useful in this situation because it can discover groups directly from the unlabelled feature data.

The motion classes may also have non-round shapes in the feature space. Different movements can create curved or hollow regions rather than simple groups centred around one average point. A density-based method is therefore a better match than a nearest-centre method such as k-means.

3 Method and Setup

The experiment used a synthetic dataset containing 1030 two-dimensional points. The dataset was generated using the fixed random seed

20250903.

It contained two crescent-shaped groups, one

ring-shaped group, and 90 uniformly scattered background points.

The dataset was divided into three parts:

- 60% training,
- 20% validation,
- 20% test.

The training set contained 618 points. It was used for DBSCAN clustering, KDE fitting, and creation of the stored lookup tables.

The validation set was used to select the KDE bandwidth. The test set was kept unused during training and tuning and was used only once at the end to evaluate the frozen model.

Both features were min–max scaled using bounds calculated only from the training set. The same bounds were used later to scale the validation, test, and query points.

DBSCAN was applied to the scaled training data using

$$\varepsilon = 0.06$$

and

$$\texttt{min_samples} = 8.$$

The baseline DBSCAN run discovered three clusters containing 209, 198, and 178 points. Another 33 training points were labelled as noise.

A Gaussian KDE was fitted separately to each discovered class. Noise points were excluded from the KDE fitting because they were not assigned to a discovered class.

The KDE bandwidth was selected using the validation set. The tested values were

$$0.03,\ 0.04,\ 0.05,\ 0.06,\ 0.08,\ 0.10,\ 0.12.$$

The highest validation mean log-likelihood was produced by

$$h = 0.05,$$

so this value was used for the final model.

Each normalised KDE surface was sampled onto a

$$16 \times 16$$

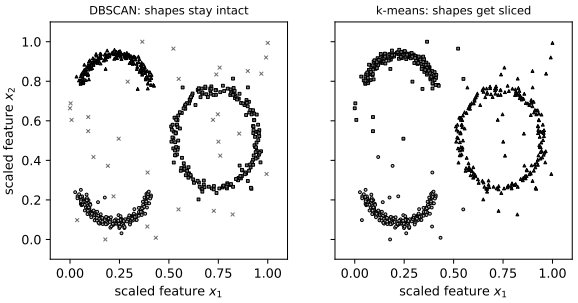


Figure 1: DBSCAN and k-means applied to the same training data.

grid and stored using `uint8`. The reject threshold was

$$\tau = 0.12.$$

4 Evidence

The evidence follows the same order as the data split. The model was built using the training data, the KDE bandwidth was selected using validation data, and the final stored model was checked using the held-out test set.

4.1 DBSCAN and k-means

Figure 1 compares DBSCAN and k-means on the same training data.

With $\varepsilon = 0.06$ and `min_samples = 8`, DBSCAN discovered three clusters containing 209, 198, and 178 points. Another 33 points were labelled as noise.

K-means was given the correct value of $k = 3$, but it still divided the non-round shapes incorrectly. The crescents were cut into centre-based regions, and the ring could not be represented as a hollow shape. This happens because k-means assigns each point to its nearest centre, while DBSCAN follows regions of connected density.

Changing ε also showed how sensitive DBSCAN is to the neighbourhood radius. With $\varepsilon = 0.03$, the scaled data produced 15 clusters and 82 noise points. With $\varepsilon = 0.30$, all 618 training points became one cluster with no noise.

The effect of feature scaling is shown in Table 1.

Table 1: Effect of scaling on DBSCAN.

Features	$\varepsilon = 0.06$	$\varepsilon = 0.30$
Raw	0 clusters, 618 noise	10 clusters, 50 noise
Scaled	3 clusters, 33 noise	1 cluster, 0 noise

Table 2: Validation bandwidth sweep.

Bandwidth h	Mean log-likelihood
0.03	1.668
0.04	1.775
0.05	1.806
0.06	1.767
0.08	1.632
0.10	1.487
0.12	1.353

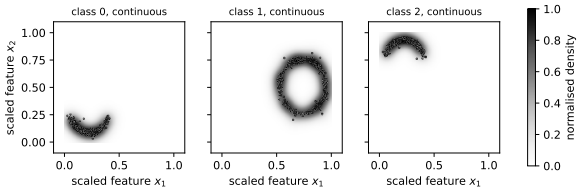


Figure 2: Continuous KDE surfaces for the three discovered classes.

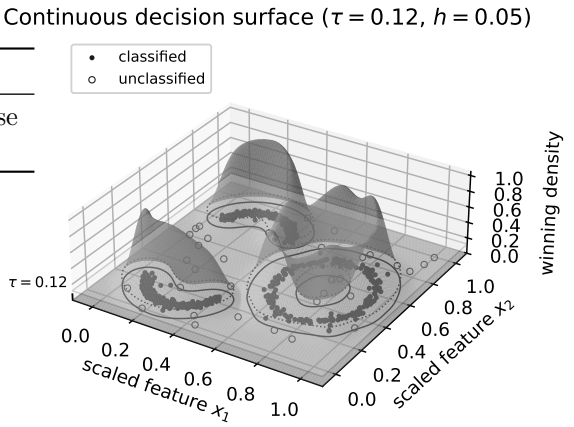


Figure 3: Continuous decision surface before discretization.

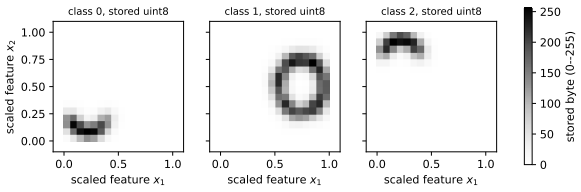


Figure 4: Per-class maps stored as uint8 lookup tables.

The same ε value therefore produces very different results in raw and scaled feature spaces. Scaling gives the distance parameter a consistent meaning across both features.

4.2 KDE and Bandwidth Selection

The KDE bandwidth was selected using the validation set. The results are shown in Table 2.

The highest validation score was obtained with $h = 0.05$, so this value was used for the final model.

Figure 2 shows the continuous per-class KDE surfaces.

The two crescent-shaped classes remain curved and the ring keeps a hollow centre. This shows that the probability maps preserve the non-round shapes discovered by DBSCAN.

Figure 3 shows the continuous decision surface before the model is reduced to the stored lookup grid.

Using a smaller bandwidth of $h = 0.03$ produced sharper and more local density regions. Using $h = 0.12$ produced smoother and wider regions and made the ring's hollow centre less distinct. This shows why the bandwidth was selected using validation data instead of by choosing the plot that looked best.

4.3 Stored Model and Memory Cost

Figure 4 shows the same class maps after sampling them onto the 16×16 grid and storing each cell as `uint8`.

The stored maps are more blocky because the grid contains only 16×16 positions, but the same general class shapes remain visible. The visible difference mainly comes from grid resolution rather than byte quantization.

The measured storage requirements are shown in Table 3.

The lookup-table footprint is

Table 3: Model footprint using a 16×16 grid.

Storage	B/cell	Per class	All 3
`uint8`	1	256	768
`uint16`	2	512	1536
`float32`	4	1024	3072
`float64`	8	2048	6144

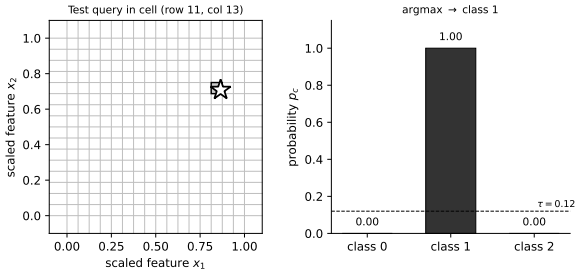


Figure 5: One held-out test point classified using the stored lookup tables.

$$\text{flash bytes} = KG^2. \tag{1}$$

For three classes and a 16×16 grid,

$$3 \times 16^2 = 768 \text{ bytes.}$$

The four scaling bounds add another 16 bytes when stored as `float32`.

The measured worst-case `uint8` error was 0.195% of peak density. This is very close to the expected maximum quantization error of approximately 0.196%. The `uint8` model therefore reduces the three lookup tables from 3072 bytes with `float32` to 768 bytes while introducing very little numerical error.

When the grid was temporarily increased to 32×32, the `uint8` storage increased from 768 bytes to 3072 bytes. The ring shape was represented more smoothly, but doubling the grid resolution in each direction increased storage by four times because the footprint grows with G^2.

4.4 Held-out Query Classification

Figure 5 shows one held-out test point being classified by the stored model.

The raw query point was

$$(5.99958893,\ 3.17868816),$$

which mapped to row 11, column 13 of the 16×16 grid.

Nearest-cell lookup read the stored values

$$[0,\ 255,\ 0]$$

for classes 0, 1, and 2. This produced a distribution of

$$[0\%,\ 100\%,\ 0\%]$$

and classified the point as class 1.

Bilinear lookup produced approximately

$$[0,\ 221.8,\ 0]$$

and gave the same 100% class 1 distribution. The two methods therefore gave the same decision for this point. Nearest-cell lookup requires K byte reads, while bilinear lookup requires $4K$ byte reads.

Around the query cell, the class 1 byte values were

$$\begin{array}{ccc} 187 & 164 & 76 \\ 208 & 255 & 183 \\ 68 & 159 & 222 \end{array}$$

while the corresponding class 0 and class 2 values were all zero. The strong change between nearby cells shows that the stored surface can change sharply at this grid resolution. Bilinear interpolation would have the greatest effect near these cell boundaries.

The reject threshold also affected a large part of the feature space. A total of 127 of the 256 grid cells were below $\tau = 0.12$, which is 49.6% of the grid. Without a reject threshold, points in these low-density areas would still be forced into one of the classes even though the model had little evidence for that decision.

The raw point at the centre of the ring,

$$(4.8,\ 1.6),$$

was classified as UNKNOWN. This is the expected behaviour because the centre is surrounded by ring data but contains no training samples itself.

Table 4: Continuous KDE versus stored uint8 decisions.

	c0	c1	c2	Unknown
Class 0	69	0	0	0
Class 1	0	68	0	0
Class 2	0	0	63	0
Unknown	0	0	0	6

4.5 Held-out Test Result

The final stored model was evaluated once on the 206 held-out test points. The stored `uint8` classifier reproduced the continuous KDE decision on

$$100.0\%$$

of the test points.

The confusion table was

All 206 decisions agreed. This includes six test points that were rejected as UNKNOWN by both the continuous and stored models. For this held-out test set, the reduction from continuous floating-point KDE surfaces to the 768-byte `uint8` lookup model did not change any classification decisions.

5 Limitations and Remaining Uncertainty

This experiment used only one synthetic two-dimensional dataset. The results show that the pipeline worked well on this dataset, but they do not prove that the same performance would be obtained with real sensor data or with a larger number of features.

The class labels were created by DBSCAN instead of being provided by an external ground-truth label set. This means that the later stages of the pipeline assume the DBSCAN clusters are meaningful classes. The figures can show that the clusters are consistent with the shapes in the dataset, but they cannot prove that those labels would be correct for a real application.

The reject threshold was set to

$$\tau = 0.12.$$

This value was selected as an engineering setting rather than being derived from labelled ground-truth data. The reject-region measurement showed that 127 of the 256 grid cells, or 49.6%, were below this threshold. This is useful because it prevents the classifier from forcing low-density points into a class, but a real application would need more evidence to choose the best threshold.

The final test result also has an important limitation. The stored `uint8` model was compared against the continuous KDE's own decisions, not against true class labels. The 100.0% agreement therefore shows that the stored model reproduced the continuous model on the 206 held-out test points. It does not prove that every decision represents the true physical class, because no ground-truth labels exist in this experiment.

The grid resolution is another trade-off. Increasing the grid from 16×16 to 32×32 improved the spatial detail of the stored maps, especially around the ring, but increased the lookup-table storage from 768 bytes to 3072 bytes. This happens because memory grows with G^2 while the resolution of each axis grows only with G. A real design would therefore need to choose the grid size based on its available flash and the amount of spatial detail required.

The KDE results were also sensitive to bandwidth. The validation sweep showed that small bandwidth values produced sharper density regions, while large values smoothed the maps and began to fill the hollow centre of the ring. Although $h = 0.05$ performed best on the validation data used here, a different dataset could require a different value.

The next step would be to test the same pipeline using real labelled sensor data from the intended application. This would make it possible to compare the DBSCAN-generated classes against known classes, measure true classification accuracy, and select the reject threshold using real errors instead of only agreement with the continuous KDE.